Домашняя самостоятельная работа: Разработка консольного приложения на Python (ООП и JSON)

Дисциплина: Программирование на Python и разработка веб-приложений

Тема: Принципы ООП (Инкапсуляция, Наследование, Полиморфизм), работа с JSON и валидация данных

Формат сдачи: .py файл или ссылка на GitHub

1. Цель работы

Закрепить знания по ООП (класс, объект, инкапсуляция, наследование, полиморфизм), научиться организовывать систему регистрации/авторизации пользователей, проводить валидацию пользовательского ввода и сохранять состояние программы в файл формата JSON.

2. Темы проектов (на выбор)

Вы можете выбрать одну из следующих предметных областей:

1. **Система заказов «Пиццерия / Ресторан»** (клиент, администратор, типы пицц/блюд, корзина, заказы).
2. **Менеджер задач (Task Tracker)** (обычный пользователь, менеджер, обычные/срочные задачи).
3. **Система бронирования номеров в отеле** (клиент, администратор, категории номеров).
4. **Онлайн-библиотека / Книжный магазин** (читатель, библиотекарь, категории книг).
5. **Любая другая предметная область** (по согласованию с преподавателем).

3. Технические требования

3.1. Принципы ООП

1. **Инкапсуляция:**
    - Все ключевые атрибуты классов (пароли, баланс, персональные данные, статусы) должны быть защищены (_attribute) или приваты (__attribute).
    - Доступ и изменение данных должны осуществляться через методы (геттеры/сеттеры или специальные методы класса, например, check_password()).
2. **Наследование:**
    - Должна быть построена иерархия классов.

- *Пример для пользователей:* Базовый класс User → Дочерние классы Client и Admin (или PremiumUser).
- *Пример для сущностей:* Базовый класс Product / Task → Дочерние классы Pizza / UrgentTask.

3. **Полиморфизм:**
    - Реализуйте переопределение методов в дочерних классах (например, метод get_info(), calculate_total() или execute(), который работает по-разному для разных типов объектов).

3.2. Авторизация и регистрация

- **Регистрация:** Создание нового пользователя с выбором его роли/типа, проверкой уникальности логина/имени.
- **Авторизация:** Вход в систему по логину и паролю. Меню действий должно меняться или ограничиваться в зависимости от вошедшего пользователя.

3.3. Валидация и проверки деталей

- Все вводы от пользователя (input()) должны быть защищены блоками try-except (обработка ValueError при переводе в int/float).
- Проверка условий (нельзя вводить отрицательные суммы, пустые строки, некорректные пункты меню).

3.4. Сохранение и загрузка данных (JSON)

- При старте программы данные должны загружаться из файла .json (например, data.json или users.json).
- Все изменения (регистрация нового пользователя, совершение заказа, изменение статусов) должны автоматически или при выходе сохраняться обратно в файл .json.
- Корректная обработка ошибки json.JSONDecodeError или отсутствия файла при первом запуске.

4. Структура консольного интерфейса

Программа должна работать в бесконечном цикле while True с меню:

Главное меню:

1. Регистрация
2. Вход
3. Завершить программу (с сохранением данных)

Меню авторизованного пользователя:

1. Выполнить основное действие (сделать заказ / добавить задачу / просмотреть каталог)
2. Пополнить баланс / Посмотреть историю
3. Показать информацию о профиле
4. Выйти из аккаунта